

Exp-1

1. WAP to implement the Perceptron Learning Algorithm using numpy in Python. Evaluate performance of a single perceptron for NAND and XOR truth tables as input dataset.

✓ Code Explanation

This Python script implements a **Perceptron Learning Algorithm** and evaluates its performance on **NAND** and **XOR** logic gates. Additionally, it visualizes the decision boundaries.

1. Importing Required Libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

- `numpy` is used for numerical operations.
- `matplotlib.pyplot` is used for visualizing the decision boundary.

2. Defining the Perceptron Class

Initialization

```
class Perceptron:
    def __init__(self, input_size, learning_rate=0.1, epochs=10):
        self.weights = np.random.randn(input_size + 1) # +1 for bias
        self.learning_rate = learning_rate
        self.epochs = epochs
```

- The **weights** are randomly initialized, including an extra bias weight.
- `learning_rate` controls how much the weights update per iteration.
- `epochs` define how many times the training process repeats.

Activation Function

```
def activation(self, x):
    return 1 if x >= 0 else 0
```

- Uses a **step function** (Heaviside function), which outputs:
 - 1 if input ≥ 0 .
 - 0 otherwise.

Prediction Function

```
def predict(self, x):
    x = np.insert(x, 0, 1) # Adding bias term
    return self.activation(np.dot(self.weights, x))
```

- The input `x` is **augmented** with a bias term.
- Computes the weighted sum using `np.dot()`, then applies the activation function.

Training the Perceptron

```
def train(self, X, y):
    for _ in range(self.epochs):
        for xi, target in zip(X, y):
            xi = np.insert(xi, 0, 1) # Adding bias term
            prediction = self.activation(np.dot(self.weights, xi))
            error = target - prediction
            self.weights += self.learning_rate * error * xi
```

- The model loops over the dataset multiple times (epochs).
- For each input `xi`:
 - **Bias** is added.
 - Computes **prediction**.

- Computes **error** (target - prediction).
- Updates weights using the **Perceptron Learning Rule**: $w = w + \eta \times \text{error} \times x$

Evaluating Accuracy

```
def evaluate(self, X, y):
    correct = sum(self.predict(x) == target for x, target in zip(X, y))
    accuracy = correct / len(y)
    return accuracy
```

- Computes accuracy by comparing predictions with actual values.

Plotting Decision Boundary

```
def plot_decision_boundary(self, X, y, title):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
    Z = np.array([self.predict(np.array([x, y])) for x, y in zip(xx.ravel(), yy.ravel())])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o')
    plt.title(title)
    plt.xlabel("Input 1")
    plt.ylabel("Input 2")
    plt.show()
```

- **Creates a grid of points** in the feature space.
- **Predicts outputs** for each point in the grid.
- Uses `contourf()` to plot regions of different predictions.
- **Scatters the original data points** for comparison.

3. Training the Perceptron on NAND

```
X_nand = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_nand = np.array([1, 1, 1, 0]) # NAND output

perceptron_nand = Perceptron(input_size=2, learning_rate=0.1, epochs=10)
perceptron_nand.train(X_nand, y_nand)
nand_accuracy = perceptron_nand.evaluate(X_nand, y_nand)
print("Perceptron NAND Accuracy:", nand_accuracy)
perceptron_nand.plot_decision_boundary(X_nand, y_nand, "NAND Perceptron Decision Boundary")
```

- The **NAND gate is linearly separable**, so the perceptron should learn it successfully.
- Expected **accuracy: 100%**.
- The **decision boundary** is plotted.

4. Training the Perceptron on XOR

```
X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_xor = np.array([0, 1, 1, 0]) # XOR output

perceptron_xor = Perceptron(input_size=2, learning_rate=0.1, epochs=10)
perceptron_xor.train(X_xor, y_xor)
xor_accuracy = perceptron_xor.evaluate(X_xor, y_xor)
print("Perceptron XOR Accuracy:", xor_accuracy)
perceptron_xor.plot_decision_boundary(X_xor, y_xor, "XOR Perceptron Decision Boundary")
```

- The **XOR gate is not linearly separable**, so a **single-layer perceptron fails** to learn it.
- Expected **accuracy: ~50%** (random guessing).
- The **decision boundary** will be incorrect.

Key Takeaways

1. **Perceptrons** can only solve **linearly separable problems** (e.g., NAND).

2. **XOR requires a multi-layer perceptron (MLP)** or a different model (e.g., Neural Networks).
3. **Visualization** helps understand how the perceptron separates data.

```
import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    def __init__(self, input_size, learning_rate=0.1, epochs=10):
        self.weights = np.random.randn(input_size + 1) # +1 for bias
        self.learning_rate = learning_rate
        self.epochs = epochs

    def activation(self, x):
        return 1 if x >= 0 else 0

    def predict(self, x):
        x = np.insert(x, 0, 1) # Adding bias term
        return self.activation(np.dot(self.weights, x))

    def train(self, X, y):
        for _ in range(self.epochs):
            for xi, target in zip(X, y):
                xi = np.insert(xi, 0, 1) # Adding bias term
                prediction = self.activation(np.dot(self.weights, xi))
                error = target - prediction
                self.weights += self.learning_rate * error * xi

    def evaluate(self, X, y):
        correct = sum(self.predict(x) == target for x, target in zip(X, y))
        accuracy = correct / len(y)
        return accuracy

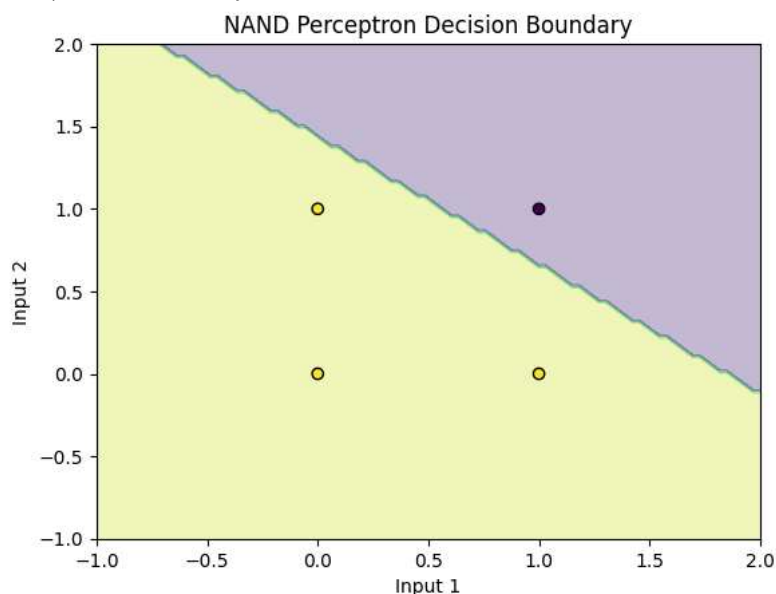
    def plot_decision_boundary(self, X, y, title):
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
        xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
        Z = np.array([self.predict(np.array([x, y])) for x, y in zip(xx.ravel(), yy.ravel())])
        Z = Z.reshape(xx.shape)

        plt.contourf(xx, yy, Z, alpha=0.3)
        plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o')
        plt.title(title)
        plt.xlabel("Input 1")
        plt.ylabel("Input 2")
        plt.show()

# NAND Truth Table
X_nand = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_nand = np.array([1, 1, 1, 0]) # NAND output

# Train Perceptron for NAND
perceptron_nand = Perceptron(input_size=2, learning_rate=0.1, epochs=100)
perceptron_nand.train(X_nand, y_nand)
nand_accuracy = perceptron_nand.evaluate(X_nand, y_nand)
print("Perceptron NAND Accuracy:", nand_accuracy)
perceptron_nand.plot_decision_boundary(X_nand, y_nand, "NAND Perceptron Decision Boundary")
```

➡ Perceptron NAND Accuracy: 1.0



```
# XOR Truth Table
X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_xor = np.array([0, 1, 1, 0]) # XOR output

# Train Perceptron for XOR
perceptron_xor = Perceptron(input_size=2, learning_rate=0.1, epochs=10)
perceptron_xor.train(X_xor, y_xor)
xor_accuracy = perceptron_xor.evaluate(X_xor, y_xor)
print("Perceptron XOR Accuracy:", xor_accuracy)
perceptron_xor.plot_decision_boundary(X_xor, y_xor, "XOR Perceptron Decision Boundary")
```

↔ Perceptron XOR Accuracy: 0.5

